PostgreSQL + Express

CSC 425 | Installation and Database Connection Lab

Name: _____ **Date:** _____

Branch: _____

Assignment purpose

Set up PostgreSQL on your computer, create a course database, and connect the provided Express server to it. Verify that an endpoint retrieves database records and returns them as JSON.

Today focuses on the connection and request/response cycle. We will examine SQL and database changes in more detail next week.

Before you begin

Use your existing course repository in VS Code. Git, Node.js 22 or later, and npm should already be installed. The instructor must publish the demo files before you begin.

Run Git and npm commands in the **repository root**: the folder containing both **client** and **server**. SQL commands go in **psql**. JavaScript in Step 8 goes in the **Node interactive prompt**.

1. Update the repository and create your branch

Open **Terminal > New Terminal** in VS Code and check your work:

```
git status
```

Commit unfinished changes on your existing work branch before continuing. Ask for help if you are unsure how to preserve them. When the working tree is clean:

```
git switch main
git pull --ff-only origin main
git switch -c firstname-lastname/postgresql-lab
```

Replace **firstname-lastname** with your name wherever it appears in this lab. Confirm that VS Code displays your new branch. If the pull fails, stop and ask for help.

Check that these files exist:

```
server/endpointExamples/studentServer.js
server/endpointExamples/setup.sql
server/.env.example
```

Checkpoint: Your personal branch is active and all three files are present.

Git and PostgreSQL store different things. Your branch stores project files. PostgreSQL stores records separately on your computer. Switching branches does not switch or reset the database.

2. Install PostgreSQL

Follow **only your operating system's instructions**. If PostgreSQL is already installed, ask the instructor to help confirm the existing setup.

Windows

1. Open the official PostgreSQL Windows download page and follow the installer link. Choose a stable release, such as PostgreSQL 18.

2. Run the installer. Include **PostgreSQL Server**, **pgAdmin 4**, and **Command Line Tools**. Keep the default installation and data folders.

3. Create and remember the administrator password for **postgres**. Keep port **5432**. Finish installation; extra Stack Builder packages are unnecessary.

4. Open **SQL Shell (psql)** from the Start menu. Enter:

Prompt	Value
Server / Database	localhost / postgres
Port / Username	5432 / postgres
Password	The administrator password you created

Installer details: EDB Windows installation guide

macOS

1. Download the stable release from Postgres.app. Move **Postgres.app** into your **Applications** folder and open it.

2. Click **Initialize** to create and start the server. Confirm that it runs on port **5432**.

3. Double-click the displayed database to open a psql terminal.

Postgres.app initially uses your macOS account name as the database username, with no database password. We will create a separate course login below. Keep Postgres.app running during the lab.

Ubuntu Linux

Open a terminal and run:

```
sudo apt update
sudo apt install postgresql postgresql-contrib
sudo systemctl enable --now postgresql
sudo -u postgres psql
```

This installs PostgreSQL, starts its service, and opens an administrator SQL session. Other Linux distributions may use different commands. Official Ubuntu installation guide

Checkpoint: You have a PostgreSQL prompt ending in **=#**. Continue on the next page.

3. Create your course database

Run the following in **psql**, not the terminal used for Git and npm. Enter each command separately.

Create the course login:

```
CREATE ROLE csc425_student LOGIN;
```

Set its password. Enter it twice when prompted and save it privately:

```
\password csc425_student
```

Create the database and assign ownership to the course login:

```
CREATE DATABASE csc425fa26 OWNER csc425_student;
```

Expected result: **CREATE DATABASE**. Run this outside any BEGIN/COMMIT block. If the role or database already exists, ask the instructor to inspect it before changing anything.

Connect to the new database; enter the course login's password if prompted:

```
\connect csc425fa26 csc425_student localhost 5432
```

Check the connection:

```
SELECT current_database(), current_user;
```

Expected database: **csc425fa26**. Expected user: **csc425_student**.

SQL statements end with a semicolon. Backslash commands such as **\connect** and **\password** do not. psql reference | CREATE DATABASE reference

4. Create the example table and sample records

In VS Code, open **server/endpointExamples/setup.sql**. Copy the entire file and paste it into your connected psql session. Press Enter to execute any remaining line.

The script creates **endpoint_examples.students** and adds five fictional students. Check the records:

```
SELECT student_id, first_name, last_name, major
FROM endpoint_examples.students
ORDER BY student_id;
```

Expect five students: **1001 Avery Morgan**, **1002 Jordan Bennett**, **1003 Taylor Reed**, **1004 Cameron Brooks**, and **1005 Riley Parker**.

Exit psql:

```
\q
```

Checkpoint: The database contains five records. Express has not started yet.

5. Configure the database connection

In VS Code, copy **server/.env.example** and name the copy **server/.env**. If that file already exists, edit it instead of replacing it.

Use these settings:

```
PORT=3000
HOST=0.0.0.0

DB_HOST=localhost
DB_PORT=5432
DB_NAME=csc425fa26
DB_USER=csc425_student
DB_PASSWORD="YOUR_COURSE_LOGIN_PASSWORD"
```

Replace **YOUR_COURSE_LOGIN_PASSWORD** with the password for **csc425_student**. Keep the surrounding double quotes.

Setting	Meaning
DB_HOST	Computer running PostgreSQL; localhost means your computer
DB_PORT	PostgreSQL's port: 5432
DB_NAME	Database used by this application
DB_USER / DB_PASSWORD	The course database login and its password

Keep .env private. Do not commit it, submit it, or include it in screenshots. From the repository root, confirm that Git ignores it:

```
git check-ignore server/.env
```

Expected output: **server/.env**. If nothing appears, ask for help before committing.

6. Install dependencies and start the demo

In your VS Code terminal, from the repository root:

```
npm ci --prefix server
npm run demo --prefix server
```

Expected startup message:

```
PostgreSQL connected. Student example:
http://localhost:3001/api/students
```

The actual message appears on one line. Leave this terminal running.

Checkpoint: PostgreSQL is on port **5432**; Express is on port **3001**.

The demo reads .env when it starts. Restart it after changing that file. Its Express port is fixed at 3001 in studentServer.js; PORT=3000 belongs to the original server.

7. Retrieve students with GET

Open this address in your browser:

```
http://localhost:3001/api/students
```

You should receive a JSON object with a **students** array containing all five records. The first student's values should match:

```
{
  "studentID": "1001",
  "firstName": "Avery",
  "lastName": "Morgan",
  "major": "Computer Science"
}
```

In **studentServer.js**, find **app.get('/api/students', ...)**. Trace the request: the browser calls the route, **pool.query(...)** queries PostgreSQL, **await** waits for the result, and **res.json(...)** sends **result.rows** to the browser.

8. Save a student with POST

Leave Express running. Open a **second VS Code terminal** and enter:

```
node
```

At the Node **>** prompt, paste this JavaScript:

```
const response = await fetch('http://localhost:3001/api/students', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    firstName: 'Casey',
    lastName: 'Diaz',
    major: 'Computer Science'
  })
})

console.log(response.status)
console.log(await response.json())
```

Expect status **201**, the message **Student created.**, and a saved student with a database-assigned ID. Exit the Node prompt with **.exit**, then refresh the browser. Casey should appear. Send this POST once; each successful request adds a record.

9. Confirm that records persist

In the Express terminal, press **Ctrl+C**. Run **npm run demo --prefix server** again and refresh the browser. Casey should still appear because PostgreSQL stores the record independently of the Node process.

Checkpoint: GET returns database records, POST adds a record, and that record survives an Express restart.

10. Record and submit your results

Create this file in VS Code, replacing firstname-lastname with your name:

```
server/endpointExamples/lab-notes-firstname-lastname.md
```

Record what you verified

Include your operating system and PostgreSQL version, branch name, database and table names, the record count before and after POST, the POST status, and whether Casey remained after the restart.

You can check the database version by running **SELECT version();** in a psql session. Do not include passwords or .env contents in your notes.

Explain the request/response cycle

Answer these questions in your own words:

1. What is the difference between PostgreSQL and Express?

2. What does pool.query(...) do?

3. Why does the route use await?

4. How does a database record become a JSON response?

5. Why does switching Git branches leave the database records unchanged?

Commit your notes and publish your branch

From the repository root, review your changes and run:

```
git status
git add server/endpointExamples/lab-notes-firstname-lastname.md
git commit -m "Document PostgreSQL connection lab"
git push -u origin firstname-lastname/postgresql-lab
```

On GitHub, open a pull request with:

Field	Value
Base	main
Compare	firstname-lastname/postgresql-lab

Provide the pull request link as directed in class. Leave merging to the instructor.

Completion checklist

[ ] PostgreSQL, the database, and the example table are ready.
[ ] Express connects; GET, POST, and persistence checks passed.
[ ] My notes are committed on my personal branch, with no credentials.
[ ] My branch is published and my pull request is open.

Troubleshooting and reference

Use the message you actually received to choose the next check. Record unresolved problems in your lab notes and ask for help.

Problem	What to check
Missing script: demo	Confirm the instructor published the demo, you pulled the class main branch, and you used npm run demo --prefix server from the repository root.
Connection refused / ECONNREFUSED	Confirm PostgreSQL is running. Check DB_HOST and DB_PORT. Windows: check the PostgreSQL service in Services. macOS: open Postgres.app and start the server. Ubuntu: check sudo systemctl status postgresql.
Password authentication failed	Use the password for csc425_student in server/.env, not a different administrator login. Restart Express after editing .env.
Database does not exist	Confirm you created csc425fa26 and spelled it correctly in server/.env.
relation "endpoint_examples.students" does not exist	Run setup.sql while connected to csc425fa26 as csc425_student.
Permission denied for table or schema	Confirm setup ran as csc425_student. Ask the instructor to check ownership; do not drop the database to fix this.
Port 3001 already in use / EADDRINUSE	Stop the other demo server before starting another copy.
Browser cannot connect	Confirm Express is still running and the address is http://localhost:3001/api/students.
psql command not found	Windows: use SQL Shell (psql) from Start. macOS: double-click the database in Postgres.app. Ubuntu: confirm the installation completed.
Git pull fails or files are missing	Stop and ask for help preserving your work. Do not force-push or reset away changes.

Keep these distinctions clear

5432: PostgreSQL database port.
3001: Express demo port.
server/.env: private local connection settings.
setup.sql: shared instructions for creating the example table and data.
Your Git branch: project files and your lab notes, not the database itself.

The instructor README describes the instructor's prepared Mac. Follow this student guide for your own installation.

Official references: Windows | macOS | Ubuntu | psql | CREATE DATABASE